

2. 웹 애플리케이션 개발

1) 정보 보여주기.

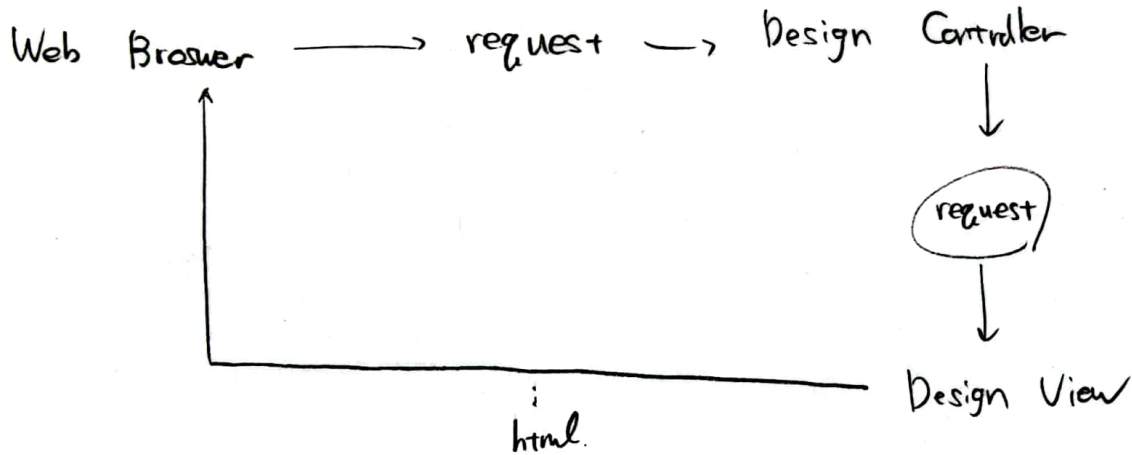
- html page : 하드 코딩 x.

- 데이터 가져오고 처리: Controller 역할 $\Rightarrow$ $\rightarrow$ View에 보여줌.

디자인 패턴 지향 $\Rightarrow$ 생성 정적 : Domain Class

View 이 전달 : Spring MVC Controller Class

View 템플릿



1-2) 도메인 설정

애플리케이션 도메인 $\Rightarrow$ 애플리케이션 이해에 필요한 개념 다룸.

~~사용자~~

데이터 타입, 이름, 식별 가능한 ID.

<MVC 흐름>

```
import lombok.Data;
import lombok.RequiredArgsConstructor;
import java.lang.reflect.Type;
```

① Data final 속성 추가 생성자 ~~getter~~ $\rightarrow$ getter/setter method x
② RequiredArgsConstructor + getter $\rightarrow$ equals(), hashCode(), toString() x) 도메인 클래스

```
public class Ingredient {
    setter 생성
    하프고 알려줌.
```

```
private final String id;
```

```
    :
    " Type type;
```

```
public static enum Type {
```

정해진 상수 값 저장, 변경 x, 메모리 사용량 ↓
Switch o.

- Taco 클래스 작성.

```
import java.util.*;  
import lombok.Data;
```

@Data

```
public class Taco {  
    private String name;  
    private List<String> ingredients;  
}
```

- 컨트롤러 클래스 생성.

Controller => Spring mvc framework = 중심적 역할.
HTTP 요청 처리 + 브라우저에 보여줄 HTML은 View에 요청.
REST 형태는 응답 용제에 데이터 추가.

<여기서는 웹 브라우저 콘텐츠 생성 위해 View 사용하는 Controller에 초점>

기능 => /design인 HTTP GET 요청 처리

=> 내역 생성

=> 데이터의 html 작성을 View 템플릿에 요청

=> 작성된 html은 웹 브라우저에 전송.

: 'Controller'

이노테이션

@Slf4j => 컴파일 시 Lombok 이 제공.

이 클래스에 자동으로 SLF4J Logger 생성.

@Controller => 해당 클래스가 Controller로 식별.

∴ Spring이 controller 클래스는 찾은 후

스프링 애플리케이션 컨텍스트에 bean으로

이 클래스의 instance 자동 생성

@RequestMapping => 클래스 수준으로 적용될 때 해당 컨트롤러가
처리하는 요청이 종류 나타냄.

ex) @RequestMapping("/design")

: /design으로 시작하는 경로에 요청
나타냄.

filterBy Type 메소드

```
private List<Ingredient> filterByType ( List<Ingredient> ingredients, Type type ) {
```

return ingredients.stream() : ingredients 리스트를 stream으로 변환.

.filter(x -> x.getType().equals(type)) : type과 일치하는 재료만 선택

.collect(Collectors.toList()); : 필터링된 재료를 새로운 리스트로 변환.

```
}
```

/ design 페이지 GET 요청

showDesignForm() 실행

① 재료 리스트 생성

② filterByType() 으로 재료 타입별로 필터링하여 모델에 추가.

③ 빈 타입 객체 생성 ... 사용자가 재료선택하기 위해

- View 디자인하기

정적 방법 => JSP, Thymeleaf, FreeMarker, Mustache, ~~groovy~~ groovy 기반

Thymeleaf => 요청 데이터 나타내는 요소 속성 추가로 갖는 html.

ex)

```
<p th:text = "${message}">place holder message</p>
```

: 요소의 몸체는 key가 message 인 세블릿 요청값으로 교체.

th:text 는 교체 수행하는 thymeleaf 네임스페이스 속성.

`${ }` : 요청속성 키 값 사용하라는 것 알려줌.

th:each => 컬렉션 반복처리, 각 요소를 하나씩 html로 나타낸다.

ex)

```
<div th:each = "ingredient : ${wrap}">
```

토르타라 의미하는 wrap 요청의 내역
value 속성값 ingredient -1 id 속성.

```
<input type="checkbox" name="ingredients" th:value = "${ingredient.id}" />
```

```
<span th:text = "${ingredient.name}">INGREDIENT</span><br/>
```

th:text 속성 이용해

↑
교체 (name 값으로)

```
</div>
```

: "wrap" 요청 속성에 있는 각 컬렉션의 각 항목에 대해 하나씩 <div> 반복.

th:each.

④ Get Mapping : /design를 HTTP GET 요청 수신될 때
요청 처리하기 showDesignForm() method 호출.

MVC 요청 - 다음 이노테이션

④ Request Mapping : 다목적 요청처리

④ Post Mapping : HTTP Post 요청처리

④ Get Mapping : HTTP Get 요청처리

④ Put Mapping : HTTP PUT 요청처리

④ Delete Mapping : HTTP DELETE 요청처리

④ Patch Mapping : HTTP PATCH 요청처리

- showDesignForm(Model model) Get 요청 들어오면 처리.
 → View(html 페이지)에서 사용할 데이터 담은 객체.

```
List<Ingredient> ingredients = Arrays.asList(  
    new Ingredient("FLTO", "Flour Tortilla", Type.WRAP)  
    ...  
);
```

① Ingredient 객체
저장하는 List 객체 생성

Type[] types = Ingredient.Type.values(); : enum Type 모든 값 가져온다.

for (Type type : types)

model.addAttribute(type.toString().toLowerCase(), filterByType(ingredient, type));
문자열로 변환 후, 소문자로 변환

model.addAttribute("taco", new Taco()); : model.addAttribute("키", "값") 형식으로
데이터 추가하면 사용 가능
return "design";

Model : 컨트롤러와 데이터 보여주는 View 사이에서 데이터 운반하는 객체.

1) 품 재를 처리

form 태그

/design 경로 POST 요청 처리하는 새로운 메소드 작성
DesignTacoController

④ Post Mapping

④ Post Mapping → /design = POST 요청 처리

```
public String processDesign(Taco design){
```

[타코 디자인 내역 저장]

```
log.info("Processing design : " + design);
```

```
return "redirect : /orders/current";
```

```
}
```

processDesign() 실행 끝난 후, 사용자 브라우저가
/orders/current 상에 경로 재접속

• form에 action 속성 선언.

form이 제출되면 폼의 모든 데이터를 모아서 폼에 나타낸 /design 로
서버에 HTTP POST 요청 전송한다.

★ log 관련 코드를 private static final org.slf4j.Logger log
= org.slf4j.LoggerFactory.getLogger(DesignTacoController.class)

/orders/current 경로 처리 Controller 필요.

④ Controller

④ Request Mapping ("/orders") ← /orders로 시작되는 경로에 요청 처리.

```
public class OrderController {
```

④ Get Mapping ("/current") ← /orders/current 경로에 GET 요청 처리.

```
public String orderForm(Model model){
```

```
model.addAttribute("order", new Order());
```

```
return "orderForm";
```

view의 이름 반환

```
}
```

```
}
```

- orderForm.html

책에서는 오류 만나지만

```
<label for = "deliveryName" > Name: </label>
<input type = "text" id = "deliveryName" th:field = "${deliveryName}" >
```

적어줘야 오류 안난다.

여기서는 (`<form th:action = "@{/orders}" ... >`) → HTTP POST 요청
액션지정

↓

```
@PostMapping
public String processOrder(Order order) {
    log.info( ... );
    return "redirect:/design";
}
```

/design으로 리다이렉트.

2.3 품 입력 유효성 검사

⇒ 입력 품 누락, 글자 제한

'유효성 검사' ⇒

- ① 메소드 if/then 블록
- ② 스프링은 자바-빈 유효성 검사 자전 (API)

라이브러리 추가.
유효성 검사 API /

구현한 Hibernate) 의존성 추가.

Spring MVC 유효성 검사

· 애플리케이션 제공

· 도메인 객체 속성 지정

검사할 클래스에 검사 규칙 선언

검사해야 할 Controller method에 검사 수행 지정

검사 예외 보여주기 품 View 수정

3-1 JSP성 검사 규칙 선언

1) Taco

: name 속성의 값이 없거나 null인지 확인. + 최소 1개 이상 선택했는 지

import javax.validation.constraints.NotNull;

jdk 8~11
: 등단.

jdk 17 이후

: jakarta.validation.constraints.NotNull

Spring Boot 3.x 부터 => Java EE → Jakarta EE

① Maven or Gradle 이 의존성 추가.

<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-validation </artifactId>

</dependency>

② import 문 작성

import jakarta.validation.constraints. (NotNull;
Size;

③ 어노테이션 작성.

⋮
@NotNull

@Size(min=5, message=" ~ ")) : 값이 null이 아님 + 최소 5개 문자 이상.
private String name;

- 다음 부분에 관한 유효성 검사. => 유효한 카드번호인가? + 길이 있나? + ...
- 배달 주소 속성 => 사용자가 입력 x 한 필드가 있는 거 없는 거 검사
- CW 속성 => 세자리수 되어야함.

L> 자바 빈 유효성 검사 API 인터페이스 + Hibernate Validator

① import jakarta.validation.constraints. {
Digits
Pattern
NotBlank

② import org.hibernate.validator.constraints.CreditCardNumber;

③ @NotBlank (message = " ~ ") =>

private String deliveryStreet;

④ @CreditCardNumber (message = " ~ ") : Luhn 알고리즘 검사에 합당한 신용카드 번호.

private String ccNumber; if) 사용가능한 지 판단: 금융망 연동.

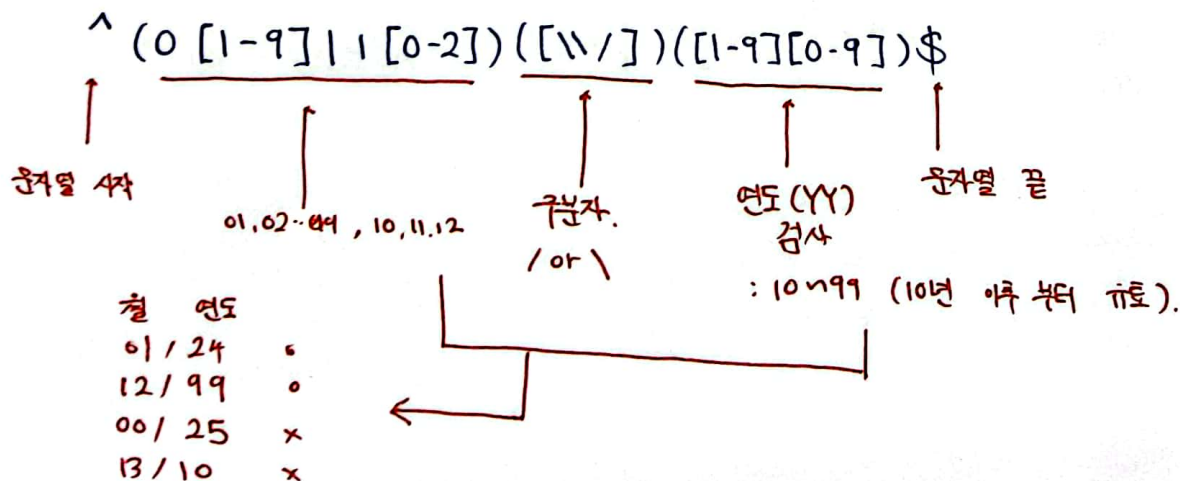
⑤ @Pattern (regexp = "^(0[1-9]|11[0-2])([\\./])([1-9][0-9])\$", message = " ~ ")

private String ccExpiration; : 정제 표현식 지정.

⑥ @Digits (integer = 3, fraction = 0, message = " ~ ") : Digits 지정하여 입력 값이 정각이

private String ccCW; 세 자리 숫자인지 검사.

<정제 표현식>



유효성 선언 → 각 폼의 POST 요청 관련 메소드에서
처리될 때 유효성 검사 수행하게 만들기.) ... @Valid.

① import 문.

```
import Jakarta.validation.Valid;
import org.springframework.validation.Errors;
```

②. 작성.

@PostMapping

```
public String processDesign (@Valid Taco design, Errors errors) {
    if (errors.hasErrors()) return "design";
```

```
    log.info(" ~ ~ ~ ");
    return "redirect:/orders/current";
}
```

제출된 Taco 객체를 유효성 검사 수행 하려고 Spring MVC에 알려줌.
제출된 폼 데이터와 Taco 객체가 바인딩 된 후, 해당 메소드 코드
실행 되기 전.

↳ 어떠한 검사라도 에러 있다면
상세 내역이 Errors 객체에 저장되고
processDesign()으로 전달.
에러 있다면 "design" 뷰 이름 반환.
hasErrors() 호출하여 에러 검사.

③ 에러 있는 장.

=> 무엇을 수정해야 하는 지 사용자가 어떻게 알까?

↳ 'th: errors <성 이용>'

<label for="ccNumber">Credit Card #: </label>

<input type="text" id="ccNumber" th:field="*{ccNumber}">

<span class="Validation Error" th:if="{#fields.hasErrors('ccNumber')}"
th:errors="*{ccNumber}">CC Num Error

hasErrors() 메소드로 ccNumber
필드에 에러 있는지 검사.

사용자 주의를 끌기 위해
에러 메시지 지정하는데 사용.

에러 0: 보여줌
에러 x: 안보임.

2.4 뷰 컨트롤러로 작업하기

- View 컨트롤러 선언하기. $\Rightarrow$ Model Data / 사용자 입력 처리하지 않는 간단한 컨트롤러.

: View에 요청을 전달하는 일만 한다.

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
```

```
import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
```

Spring 설정 클래스 작성,

② Configuration $\rightarrow$ View Controller 역할 수행하는 구성 클래스. * WebConfigurer 구현 *

```
public class WebConfig implements WebConfigurer {
```

: WebConfigurer 인터페이스 Spring MVC 구성하는 메소드 정의한다.

② Override

$\rightarrow$ 하나 이상의 뷰 컨트롤러를 등록하기 위해 ViewControllerRegistry를 인자로 받음

```
public void addViewControllers(ViewControllerRegistry registry) {
```

```
    registry.addViewController("/").setViewName("home");
```

$\rightarrow$ 객체를 통해 URL과 View 매핑.

```
}
```

" / " 요청들이오면



home 이라는 View 보여주도록한다.

2.5 View 템플릿 라이브러리 선택

- FreeMarker

: spring-boot-starter-freemarker

- Groovy Template

: " - groovy-templates

- Java Server Page

: X (Tomcat or Jetty 설정된 컨테이너 자체 제공)

- Mustache

: spring-boot-starter-mustache

- Thymeleaf

: " - thymeleaf

JSP 선택시 고려사항.

내장된 Tomcat, Jetty Container를 포함해서 Java servlet container는 /WEB-INF 밑에서 JSP코드를 찾음.

But. 실행 가능한 JAR코드를 생성 시, 외부로 충족시킬 방법 x.

∴ WAR 패키지로 생성하고 서블릿 컨테이너에 실행하는 경우에 JSP 선택.

(1) JAR 패키지로 생성

=> thymeleaf, FreeMarker ...

- 템플릿 캐싱.

: 기본적으로 템플릿은 최초 사용될 때 1번만 파싱된다.
= 코드 분석.

파싱결과 => 향후 사용하며 캐시에 저장.

But. 개발 시 약간씩 페이지 수정 => 새로고침해도 수정 전 페이지 볼 수 있음.

↳ 템플릿의 캐싱 속성 false로 바꾸기.

=> Spring boot나 DevTool 사용이 훨씬 쉽다.

'Spring MVC' -> spring web application = Web FrontEnd 개발에 사용.

